

Trabajo Práctico N° 2: Estructuras de Datos y Utilidades para Ingeniería

Consigna General: CCrear el archivo `tp2_estructuras.py`. Cada ejercicio debe estar identificado con el encabezado `print("\n* * * Ejercicio X - [Tema] * * *")`. Buscamos código limpio, eficiente y con mentalidad de procesamiento de datos.

Ejercicio 1 - Gestión de Estados (None)

- **Contexto:** En bases de datos, es vital distinguir entre un valor "cero" y un valor "nulo".
 - **Tarea:** Crear `sensor_data` inicializada en `None`. Pedir al usuario una lectura numérica.
 - **Lógica:** * Si la entrada es vacía, `sensor_data` sigue siendo `None`.
 - Si ingresa un número, convertirlo a `float`.
- **Salida:** Usar un `if` para mostrar "Error: Sensor fuera de línea" si es `None`, o "Lectura recibida: [valor]" si tiene datos.

Ejercicio 2 - Análisis de Muestras (Listas)

- **Tarea:** Generar una lista de 10 lecturas de temperatura aleatorias (entre 15 y 40) usando `random.uniform()`.
- **Mostrar:**
 - Lista completa redondeada a 1 decimal.
 - Lista ordenada de menor a mayor.
 - Promedio de la muestra (Suma / Longitud).
 - Cantidad de lecturas que superan los 30 grados (identificación de alertas).

Ejercicio 3 - Matrices de Datos (List Comprehension)

- **Contexto:** Las imágenes y tablas de datos se representan como matrices (listas de listas).
- **Tarea:** Generar una matriz de 3x3 que represente una "Grilla de Sensores" con valores iniciales en 0, pero usando **List Comprehension** anidada.
- **Extra:** Modificar el elemento central (1,1) a un valor de 100 y mostrar la matriz fila por fila.

Ejercicio 4 - Inmutabilidad de Metadatos (Tuplas)

- **Contexto:** Hay datos que nunca deben cambiar (coordenadas GPS, constantes).
- **Tarea:** Crear una tupla `punto_geografico` con (`latitud`, `longitud`, `altitud`).
- **Acción:** Intentar cambiar la altitud y capturar el error (o explicar en un comentario qué excepción lanza Python y por qué esto protege la integridad de los datos).

Ejercicio 5 - Depuración de Registros (Sets)

- **Contexto:** Eliminar duplicados y encontrar cruces de información.
- **Dadas dos listas:** `clientes_compras_web` y `clientes_compras_local`.
- **Tarea:** Convertirlas a sets y reportar:
 - **Unión:** Total de clientes únicos.
 - **Intersección:** Clientes que compran en ambos canales (Fidelizados).
 - **Diferencia:** Clientes que solo compran en la web.

Ejercicio 6 - Perfiles de Datos (Diccionarios)

- **Tarea:** Crear un diccionario `producto` con `id`, `nombre`, `precio` y `categoría`.
- **Operaciones:**
 - Actualizar el precio (aplicar un 10% de aumento).
 - Agregar la clave `stock`.
 - Iterar usando `.items()` para mostrar un reporte elegante: `CLAVE: VALOR`.

Ejercicio 7 - Cruce de Tablas (`zip`)

- **Contexto:** Tienes una lista de sensores y otra de localizaciones.
- **Tarea:** Unirlas con `zip()` para crear un diccionario donde la clave sea el nombre del sensor y el valor su ubicación. Mostrar el resultado final.

Ejercicio 8 - Control de Flujo Avanzado (`match`)

- **Tarea:** Crear un menú para un sistema de archivos de datos:
 1. Cargar CSV
 2. Exportar JSON
 3. Limpiar Datos
- **Lógica:** Usar `match` para capturar la opción del usuario. Debe incluir un caso `_` para opciones no válidas.

Ejercicio 9 - Timestamps de Procesamiento (`datetime`)

- **Contexto:** Registrar exactamente cuándo se procesó un dato.
- **Tarea:** Obtener la fecha y hora actual.
- **Formato:** Mostrarla en formato humano: `DD/MM/YYYY HH:MM`.
- **Cálculo:** Calcular cuántos días faltan para una "Fecha de Entrega de Proyecto" definida por ti (ej: 21 de septiembre).

Ejercicio 10 - Simulación de Dataset (Integrador)

- **Contexto:** Generar datos sintéticos para probar un algoritmo.

Diplomatura en Python para la Ciencia de Datos y Machine Learning

- **Tarea:** Crear una lista llamada `dataset` que contenga 5 diccionarios. Cada diccionario debe representar un `log_evento` con:
 - `timestamp` (hora actual).
 - `evento_id` (aleatorio entre 1000 y 9999).
 - `nivel` (elegir aleatoriamente de una lista: ["INFO", "WARNING", "ERROR"]).
 - **Análisis final:**
 - Filtrar y mostrar solo los eventos de nivel "ERROR" usando comprensión de listas.
 - Contar cuántos eventos se generaron en total.
-

Forma de Entrega

- Comprimir en un archivo `apellido_nombre_tp2.zip`.
- **Contenido:** Únicamente el archivo `tp2_estructuras.py`
- No se aceptan capturas de pantalla.
- El archivo debe:
 - Ejecutar sin errores.
 - Estar correctamente indentado.
 - Calidad de código: Se evaluará el uso de nombres de variables descriptivos (evitar `x`, `y`, `lista1`).
 - Tener comentarios separando cada ejercicio.